1. Bubble Sort (Método da Bolha)

- **Entrada inicial:** `[4, 2, 1, 3]`

Passada (`i`)	Índice (`j`)	Elementos Comparados (`arr[j]` e `arr[j+1]`)	Condição `arr[j] > arr[j+1]`?	Estado do Array após a operação	Comparações (Acumulado)	Trocas (Acumulado)
`i = 0`	`j = 0`	`4` e `2`	Verdadeiro (`4 > 2`)	`[2, 4, 1, 3]`	1	1
`i = 0`	`j = 1`	`4` e `1`	Verdadeiro (`4 > 1`)	`[2, 1, 4, 3]`	2	2
`i = 0`	`j = 2`	`4` e `3`	Verdadeiro (`4 > 3`)	`[2, 1, 3, 4]`	3	3
`i = 1`	`j = 0`	`2` e `1`	Verdadeiro (`2 > 1`)	`[1, 2, 3, 4]`	4	4
`i = 1`	`j = 1`	`2` e `3`	Falso (`2 > 3`)	`[1, 2, 3, 4]`	5	4
`i = 2`	`j = 0`	`1` e `2`	Falso (`1 > 2`)	`[1, 2, 3, 4]`	6	4 *(Loop encerra)*

- **Total final:** 6 comparações | 4 trocas

2. Selection Sort (Ordenação por Seleção)

- **Entrada inicial:** `[4, 2, 1, 3]`

Passada (`i`)	`min_idx` inicial	Índice analisado (`j`)	Elementos Comparados (`arr[j] < arr[min_idx]`)	Novo `min_idx`?	Estado do Array / Troca	Comp. (Acum.)	Trocas (Acum.)
`i = 0`	`0` (valor 4)	`j = 1`	`2 < 4` (Sim)	`min_idx = 1`	-	1	0

`i = 0`	`1` (valor 2)	`j = 2`	`1 < 2` (Sim)	`min_idx = 2`	-	2	0
`i = 0`	`2` (valor 1)	`j = 3`	`3 < 1` (Não)	Mantém 2	Troca `arr[0]` (4) com `arr[2]` (1) → `[1, 2, 4, 3]`	3	1
`i = 1`	`1` (valor 2)	`j = 2`	`4 < 2` (Não)	Mantém 1	-	4	1
`i = 1`	`1` (valor 2)	`j = 3`	`3 < 2` (Não)	Mantém 1	Sem troca necessária (`min_idx == i`)	5	1
`i = 2`	`2` (valor 4)	`j = 3`	`3 < 4` (Sim)	`min_idx = 3`	Troca `arr[2]` (4) com `arr[3]` (3) → `[1, 2, 3, 4]`	6	2

- **Total final:** 6 comparações | 2 trocas

3. Insertion Sort (Ordenação por Inserção)

- **Entrada inicial:** `[4, 2, 1, 3]`

`i` (Chave)	Valor da Chave (`arr[i]`)	`j` inicial	Condição `arr[j] > chave`?	Ação (Deslocamento / Inserção)	Estado do Array	Comp. (Acum.)	Trocas (Acum.)
`i = 1`	`2`	`0` (`arr[0]=4`)	`4 > 2` (Verdadeiro)	Desloca `4` para a frente; insere `2` na pos 0	`[2, 4, 1, 3]`	1	1

`i = 2`	`1`	`1` (`arr[1]=4`)	`4 > 1` (Verdadeiro)	Desloca `4`; `j` vai para 0 (`arr[0]=2`)	`[2, _, 4, 3]`	2	2
`i = 2`	`1`	`0` (`arr[0]=2`)	`2 > 1` (Verdadeiro)	Desloca `2`; `j` vai para -1; insere `1` na pos 0	`[1, 2, 4, 3]`	3	3
`i = 3`	`3`	`2` (`arr[2]=4`)	`4 > 3` (Verdadeiro)	Desloca `4`; `j` vai para 1 (`arr[1]=2`)	`[1, 2, _, 4]`	4	4
`i = 3`	`3`	`1` (`arr[1]=2`)	`2 > 3` (Falso)	Encerra while; insere `3` na pos `j+1` (2)	`[1, 2, 3, 4]`	5	4

- **Total final:** 5 comparações | 4 trocas (movimentações)

4. Merge Sort (Ordenação por Mesclagem)

- **Entrada inicial:** `[4, 2, 1, 3]`

1. **Divisão (Divide-and-Conquer):**
   - Divide `[4, 2, 1, 3]` →\rightarrow→ `[4, 2]` e `[1, 3]`
   - Divide `[4, 2]` →\rightarrow→ `[4]` e `[2]`
   - Divide `[1, 3]` →\rightarrow→ `[1]` e `[3]`
2. **Mesclagem (Merge):**
   - Mescla `[4]` e `[2]`: Compara `4` e `2` (1ª comp). Menor é `2`, depois `4`. Resultado: `[2, 4]` (2 cópias/trocas).
   - Mescla `[1]` e `[3]`: Compara `1` e `3` (2ª comp). Menor é `1`, depois `3`. Resultado: `[1, 3]` (2 cópias/trocas).
   - Mescla final entre `[2, 4]` e `[1, 3]`:
     - Compara `2` e `1` (falso) →\rightarrow→ pega `1`. (3ª comp, 1ª cópia)
     - Compara `2` e `3` (verdadeiro) →\rightarrow→ pega `2`. (4ª comp, 2ª cópia)
     - Compara `4` e `3` (falso) →\rightarrow→ pega `3`. (5ª comp, 3ª cópia)
     - Resta o `4`, que entra direto na última posição. (4ª cópia)
     - Resultado final: `[1, 2, 3, 4]`

- **Total final:** 5 comparações | 8 movimentações/cópias

5. Quick Sort (Ordenação Rápida)

- **Entrada inicial:** `[4, 2, 1, 3]`

- **Particionamento Inicial (`baixo=0`, `alto=3`):**
  - Pivô escolhido = `arr[3] = 3`. `i = -1`.
  - `j = 0`: Compara `4 <= 3`? Falso. (1ª comp)
  - `j = 1`: Compara `2 <= 3`? Verdadeiro. `i` vira `0`. Troca `arr[0]` (4) com `arr[1]` (2) →\rightarrow→ `[2, 4, 1, 3]`. (2ª comp, 1ª troca)
  - `j = 2`: Compara `1 <= 3`? Verdadeiro. `i` vira `1`. Troca `arr[1]` (4) com `arr[2]` (1) →\rightarrow→ `[2, 1, 4, 3]`. (3ª comp, 2ª troca)
  - Fim do loop `j`. Posiciona o pivô (3) trocando `arr[i+1]` (`arr[2] = 4`) com `arr[alto]` (3) →\rightarrow→ `[2, 1, 3, 4]`. (3ª troca)
  - Pivô posicionado no índice `2` (`valor 3`).
- **Sub-array esquerdo (`[2, 1]`):**
  - Pivô = `1`, `i = -1`.
  - `j = 0`: Compara `2 <= 1`? Falso. (4ª comp)
  - Coloca pivô (1) no lugar certo trocando com `arr[0]` (2) →\rightarrow→ `[1, 2, 3, 4]`. (4ª troca)
- **Sub-array direito (`[4]`):** Já está ordenado (tamanho 1).
- **Total final:** 4 comparações | 4 trocas